

Machine Learning Models for Understanding Hotel Review Sentiments: An Evaluation with Weka

Introduction

In the hospitality and service industry, customer feedback is crucial for shaping business strategies and improving customer satisfaction. Online platforms like TripAdvisor provide a wealth of customer reviews that offer insights into various aspects of service. This report aims to analyze a dataset of TripAdvisor hotel reviews to gain insights that can inform business strategies for a company interested in understanding customer sentiment from Facebook reviews. The objective is to develop machine learning models using the TripAdvisor dataset to predict hotel ratings or specific service attributes based on review texts. Additionally, the report will gauge the influence of reviews and their authors, both online and in real life, to provide comprehensive insights into customer feedback.

This report will address tasks 1. and 4.

Data Description

The dataset, named "assignment_data.arff," consists of 3,118 TripAdvisor hotel reviews, detailed across 22 attributes. It encompasses a mix of numeric ratings on various aspects of hotel service (like cleanliness, service quality, and location) and qualitative feedback through review texts and titles. Notably, it includes metrics of review influence such as the number of helpful votes, and information about the authors, like their travel history and online influence. Preliminary examination reveals a diverse range of reviews, highlighting varied customer experiences and perceptions. The inclusion of both numeric ratings and textual reviews allows for a multi-dimensional analysis.

Task 1: Predicting Hotel Ratings from Review Text

Data preprocessing

During the preprocessing phase, we refined the dataset to concentrate on attributes directly relevant to predicting the hotel's overall ratings from review texts. All attributes except Rating_overall, text, and title were removed.

Reasons for Selection:

- **Rating_overall:** This is our target variable, representing the numerical rating customers give to a hotel. It's essential for our predictive modeling task.
- **Text:** The main body of the review is rich in detailed customer feedback, crucial for understanding the factors influencing overall ratings.
- **Title:** Often summarizing the review's sentiment, the title can offer additional predictive value, capturing the essence of customer feedback in a few words.

Rating_overall was transformed to nominal using Weka's **NumericToNominal** filter. This was done, since nominal values allow the use of classification algorithms in Weka.

There were some missing values in **Rating_overall** which were replaced using **ReplaceMissingValues** filter.

Text and **title** were transformed to word vectors using the **StringToWordVector** filter in Weka, using LovinsStemmer, MultiStopwords, IDF and TF- transforms and minTermFreq=5. These configurations allow the text to be better used in machine learning tasks, by reducing words to their root form and removing common words that might not be useful for the analysis.

After preprocessing there are 1005 attributes including Rating_overall and the filtered words.

Model Selection

The models being tested are J48 Decision Tree, Random Forest and Neural Network.

- J48 Decision Tree offers a straightforward, interpretable model that easily handles categorical data, making it a good starting point for analysis.
- Random Forest improves upon single decision trees by providing higher accuracy and better generalization.
- Neural Network is capable of capturing complex patterns and interactions in the data, which is ideal for the nuanced task of text analysis.

Training Approach

In this analysis, a 66% percentage split training option was selected due to the extensive computational resources required for cross-validation with neural networks, which can be time-prohibitive. This approach allows for a balance between training and testing the models while maintaining a reasonable computation time.

Feature Selection

Using the InfoGainAttributeEval evaluator and Ranker search method in Weka's "Select attributes" tab, an initial feature selection process identified approximately 200 features providing informational gain. The initial models were trained using this subset of features to maximize the efficiency of the attribute space.

J48 Decision Tree

Initial Testing:

- Settings: Default, 200 features via AttributeSelectedClassifier.
- Results: The model built in 4.36 seconds with 38.4906% of instances correctly classified (408 instances).

Further Testing and Adjustments:

- Unpruned trees did not yield an improvement.
- Increasing the minNumObj parameter to 7 and the feature count to 250 led to better performance.

Optimized Model:

- Parameters: minNumObj=7, 250 features.
- Results: The model built in 4.54 seconds with 42.9245% of instances correctly classified (455 instances).

```
Time taken to build model: 4.54 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0.03 seconds

=== Summary ===

Correctly Classified Instances      455      42.9245 %
Incorrectly Classified Instances    605      57.0755 %
Kappa statistic                     0.1983
```

```
=== Confusion Matrix ===

  a  b  c  d  e  <-- classified as
30 10 11 23  8 |  a = 1
 9 15 10 28 13 |  b = 2
 6 14 42 90 31 |  c = 3
14 15 44 184 109 | d = 4
 5  9 22 134 184 |  e = 5
```

Random Forest

Initial Testing:

- Settings: Default, 200 features.
- Results: The model built in 4.89 seconds with 45.8491% of instances correctly classified (486 instances).

Further Testing and Adjustments:

- Increasing numIterations to 150 and the feature count to 250 led to improved performance.

Optimized Model:

- Parameters: numIterations=150, 250 features.
- Results: The model built in 7.54 seconds with 47.1698% of instances correctly classified (500 instances).

```

Time taken to build model: 7.54 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0.2 seconds

=== Summary ===

Correctly Classified Instances      500      47.1698 %
Incorrectly Classified Instances    560      52.8302 %
Kappa statistic                     0.234

=== Confusion Matrix ===

  a   b   c   d   e  <-- classified as
38  10   4  21   9 |   a = 1
14   5   6  39  11 |   b = 2
 8   4  17 127  27 |   c = 3
 6   0  20 227 113 |   d = 4
 1   0   9 131 213 |   e = 5

```

Neural Network (Multilayer Perceptron)

Initial Testing:

- Settings: Default, 200 features.
- Results: The model took 215.79 seconds to build, with 42.6415% of instances correctly classified (452 instances).

Further Testing and Adjustments:

- Due to significant build times, limited testing was conducted with a reduced trainingTime.
- Variations in network architecture and learning parameters were explored to refine performance.

Optimized Model:

- Parameters: trainingTime=10, hiddenLayers=i (where i equals the number attributes), learningRate=0.1, momentum=0.4.
- Results: The model built in 15.93 seconds with 46.9811% of instances correctly classified (498 instances).

```

Time taken to build model: 15.93 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0.25 seconds

=== Summary ===

Correctly Classified Instances      498           46.9811 %
Incorrectly Classified Instances    562           53.0189 %
Kappa statistic                    0.2745

=== Confusion Matrix ===

  a   b   c   d   e   <-- classified as
35  27  10   3   7 |   a = 1
14  24  16   9  12 |   b = 2
11  23  61  48  40 |   c = 3
11  14  54 113 174 |   d = 4
 7   4  26  52 265 |   e = 5

```

J48 Decision Tree Analysis

- **Performance:** The optimized J48 model achieved a classification accuracy of 42.9245%. While this reflects some predictive capability, more than half of the instances were incorrectly classified.
- **Kappa Statistic:** At 0.1983, the Kappa statistic indicates a slight agreement beyond chance, which could be considered low, suggesting room for improvement.
- **Confusion Matrix:** The confusion matrix shows some ability to correctly classify instances, but significant confusion is evident, especially among classes c, d, and e. This confusion is likely due to overlapping features that are not well separated by the decision boundaries in a J48 model.

Random Forest Analysis

- **Performance:** With a slightly improved accuracy of 47.1698%, the Random Forest model outperforms the J48, likely due to its ensemble nature, which typically handles variance and bias better.
- **Kappa Statistic:** The Kappa of 0.234 indicates a fair agreement but still suggests considerable room for improvement.
- **Confusion Matrix:** This model appears to classify certain classes (like 'e') with higher accuracy, but confusion remains, particularly for the middle classes. The ensemble nature of Random Forest helps to mitigate some of the overfitting issues present in single decision trees.

Neural Network Analysis

- **Performance:** The Neural Network achieved 46.9811% accuracy, a marked improvement over the J48 and comparable to the Random Forest, even with a significantly reduced training time.

- **Kappa Statistic:** The Kappa statistic of 0.2745 is the highest among the three models, suggesting a better agreement beyond chance compared to the others.
- **Build Time:** Despite being the most computationally expensive model initially, adjustments to the training time significantly reduced the build time while maintaining a relatively high classification accuracy.
- **Confusion Matrix:** The network showed a strong performance on the 'e' class but still struggled with distinguishing between other classes, similar to the other models.

Overall Analysis

- All models show challenges with classification, as indicated by accuracy rates below 50% and the considerable misclassifications evident in the confusion matrices. This suggests a possible overlap in features between classes or a lack of distinctive features to differentiate the classes effectively.
- The Random Forest model displayed the best balance between accuracy and computational time, making it a potentially suitable model for this particular dataset and task when considering both efficiency and performance.
- The Neural Network's performance was notable given the reduced training time, but the initial longer build time may be a drawback in scenarios where rapid model building is required.
- Considering the overall model performance and computational demands, the Random Forest seems to be the most promising approach, achieving the highest accuracy with a reasonable build time. However, given the generally moderate performance of all models, further investigation into data quality, additional feature engineering, and alternative modeling approaches is recommended to improve predictive accuracy.

Task 4: Predicting Author Influence Online

Data preprocessing

- **Hotel_id**, **id**, and **author_id** were removed from the dataset. These attributes represent unique identifiers that do not contribute predictive power to the model as they are specific to each instance and do not generalize across the dataset.
- **Author_username**, **Author_location**, **title** and **text** were excluded due to their textual nature. Given that the task at hand does not involve text mining, these string-based features would not be utilized in the models.
- There were some missing values in **Author_num_helpful_votes** which were replaced using **ReplaceMissingValues** filter.
- Applied the **MathExpression** filter in Weka, using the function **ifelse(A<100,1,2)**. This function was used to transform the **Author_num_helpful_votes** attribute, which quantifies the number of helpful votes an author has received for their reviews. By applying this conditional transformation, authors are divided into two groups: those with fewer than 100 helpful votes (assigned a value of 1) and those with 100 or

more helpful votes (assigned a value of 2). This binary classification aligns with the business interest in distinguishing authors with significant online influence.

- **Author_num_helpful_votes** was then transformed to nominal using Weka's **NumericToNominal** filter. This was done, since nominal values allow the use of classification algorithms in Weka.

Model Selection

Model selection and training approach will be the same as in task 1.

Feature Selection

Using the InfoGainAttributeEval evaluator and Ranker search method in Weka's "Select attributes" tab, an initial feature selection process identified two informative attributes in the context of our task: Author_num_cities and Author_num_reviews.

average merit	average rank	attribute
0.094 +- 0.003	1 +- 0	13 Author_num_cities
0.089 +- 0.003	2 +- 0	15 Author_num_reviews
0.007 +- 0.001	3 +- 0	1 num_helpful_votes
0.004 +- 0.001	4.4 +- 0.66	3 revisit
0.003 +- 0.001	5.9 +- 1.45	11 Rating_check_in_front_desk
0.003 +- 0	6.2 +- 0.6	2 via_mobile
0.002 +- 0	7 +- 1.18	4 Rating_overall
0.001 +- 0.002	7.4 +- 1.2	5 Rating_service
0 +- 0	9.2 +- 0.4	6 Rating_cleanliness
0.001 +- 0.002	9.7 +- 2.61	9 Rating_sleep_quality
0 +- 0	10.2 +- 0.4	10 Rating_rooms
0 +- 0	12.2 +- 0.4	8 Rating_location
0 +- 0	12.8 +- 0.4	12 Rating_business_service_(e_g_internet_access)
0 +- 0	14 +- 0	7 Rating_value

These two features will be used in the initial round of testing for each model.

J48 Decision Tree

Initial Testing:

- Settings: Default, 2 features via AttributeSelectedClassifier.
- Results: The model built in 0.01 seconds with 97.6415% of instances correctly classified (1035 instances).

Further Testing and Adjustments:

- An unpruned tree had no significant effect on performance.
- Increasing the minNumObj parameter resulted in diminished accuracy.

- Altering the number of features, either increasing or decreasing, also led to a decline in performance.

Optimized Model:

- Same as initial

```
Time taken to build model: 0.01 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0 seconds

=== Summary ===

Correctly Classified Instances      1035      97.6415 %
Incorrectly Classified Instances     25      2.3585 %
Kappa statistic                     0.534

=== Confusion Matrix ===

   a    b  <-- classified as
1020    6 |    a = 1
   19   15 |    b = 2
```

Random Forest

Initial Testing:

- Settings: Default, 2 features.
- Results: The model built in 0.27 seconds with 97.7358% of instances correctly classified (1036 instances).

Further Testing and Adjustments:

- Modifications to the number of iterations, both increases and decreases, led to worse performance outcomes.
- Variations in the number of features also resulted in decreased accuracy.

Optimized Model:

- Same as initial


```

Time taken to build model: 0.27 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0.06 seconds

=== Summary ===

Correctly Classified Instances      1036      97.7358 %
Incorrectly Classified Instances      24      2.2642 %
Kappa statistic      0.5886

=== Confusion Matrix ===

   a    b  <-- classified as
1018    8 |    a = 1
   16   18 |    b = 2

```

Neural Network (Multilayer Perceptron)

Initial Testing:

- Settings: Default, 2 features.
- Results: The model took 0.3 seconds to build, with 96.8868% of instances correctly classified (1027 instances).

Further Testing and Adjustments:

- Reducing the learning rate to 0.08 showed a positive impact, enhancing accuracy.
- Adjustments to the momentum parameter had a negligible effect.
- Enlarging the validation set size and varying the training time negatively affected performance.
- Changes to the number of hidden layers had minimal impact.

Optimized Model:

- Parameters: learningRate=0.08
- Results: The model built in 0.29 seconds with 97.3585% of instances correctly classified (1032 instances).

```

Time taken to build model: 0.29 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0 seconds

=== Summary ===

Correctly Classified Instances      1032      97.3585 %
Incorrectly Classified Instances     28      2.6415 %
Kappa statistic                     0.4871

=== Confusion Matrix ===

   a    b  <-- classified as
1018    8 |    a = 1
   20   14 |    b = 2

```

J48 Decision Tree Analysis

- **Performance:** Exceptional performance with an accuracy of 97.6415%. This high accuracy suggests that the two features used are highly predictive of the class labels.
- **Kappa Statistic:** A Kappa statistic of 0.534 indicates moderate agreement, which is quite good for such a simple model.
- **Confusion Matrix:** The confusion matrix reveals that the model is highly effective at identifying class 'a', but less so for class 'b', with more false negatives than false positives. This could be an indication of a class imbalance issue or could reflect that the model is more conservative in predicting the less frequent class.

Random Forest Analysis

- **Performance:** Slightly higher accuracy than J48 at 97.7358%, showing Random Forest's strength in handling variance and bias in the data.
- **Kappa Statistic:** The Kappa statistic improved to 0.5886, which signifies good predictive strength beyond chance agreement.
- **Confusion Matrix:** Similar to the J48 model, Random Forest performs well for class 'a' but shows some limitations with class 'b', which is the minority class. This pattern suggests the model may be biased towards the majority class.

Neural Network Analysis

- **Performance:** A strong accuracy of 97.3585%, with the optimized Neural Network performing comparably to the J48 and Random Forest models.
- **Kappa Statistic:** At 0.4871, the Kappa statistic is slightly lower than the other two models but still indicates moderate agreement.
- **Confusion Matrix:** The model demonstrates a similar pattern to the others, with high accuracy for class 'a' and some difficulty in correctly classifying class 'b'.

General Observations

- All models achieved a high level of accuracy with the given features, which underscores the importance of Author_num_cities and Author_num_reviews in predicting the helpfulness votes an author receives.
- The consistency across models suggests that the two features provide robust signals for the predictive task.
- The confusion matrices for all models reveal a challenge in correctly classifying the minority class 'b'. This could be indicative of a need for techniques to address class imbalance, such as oversampling the minority class or adjusting the classification threshold.
- Given the high accuracy and the low build and test times, all models are potentially suitable for real-time predictions in a production environment.
- The Random Forest model had the highest Kappa statistic, which may make it the most reliable model among the three in practice, assuming the costs of false positives and false negatives are similar.

SMOTE for Imbalanced Samples

Using SMOTE to increase the amount of 'b' samples by 3000%

J48 Decision Tree Model with SMOTE

- **Accuracy:** Decreased to 94.0849%
- **Kappa Statistic:** A high Kappa statistic of 0.8813 reflects excellent agreement beyond chance, suggesting that the model can distinguish between classes 'a' and 'b' very effectively.
- **Confusion Matrix:**

```
=== Confusion Matrix ===
      a    b   <-- classified as
989  57 |    a = 1
 60 872 |    b = 2
```

The increase in true positives for class 'b' (from 15 to 872) and the balanced number of false positives and false negatives demonstrate that the model has become much more effective at classifying the minority class without compromising the majority class.

Random Forest Model with SMOTE

- **Accuracy:** The accuracy saw a slight decrease to 96.9161%
- **Kappa Statistic:** An outstanding Kappa statistic of 0.9381 indicates a very strong predictive performance.
- **Confusion Matrix:**

```

=== Confusion Matrix ===
      a      b  <-- classified as
1020   26 |    a = 1
   35  897 |    b = 2

```

The matrix shows a high number of true positives for both classes with relatively few misclassifications, implying that the model has benefited significantly from the SMOTE application.

Neural Network Model with SMOTE

- **Accuracy:** The accuracy is lower compared to the other two models at 88.1193%
- **Kappa Statistic:** The Kappa statistic of 0.7622, while lower than the tree-based models, still indicates substantial agreement.
- **Confusion Matrix:**

```

=== Confusion Matrix ===
      a      b  <-- classified as
 908 138 |    a = 1
   97 835 |    b = 2

```

There is a noticeable number of misclassifications for both classes, suggesting that while the Neural Network has improved in identifying the minority class, it has also become more prone to misclassification errors compared to the other models. The problems are most likely because of poor tuning, since the same settings were used again even with the balanced samples.

The implementation of SMOTE has led to a significant increase in the ability of all models to identify the previously underrepresented class 'b', with marked improvements in the Kappa statistic. The Random Forest model appears to be the most robust, yielding the highest accuracy and Kappa statistics, suggesting that this model benefits the most from a balanced class distribution.